

Historia de usuario M6.1S2

Persistencia Estratégica de "Eventify" - De la Memoria a la Base de Datos

Objetivo de la Historia de usuario

Evolucionar "**Eventify**" integrando Spring Data JPA e Hibernate para gestionar la persistencia real, implementando un CRUD completo y optimizando el manejo de datos mediante Paginación y Ordenamiento.

Contexto de Negocio

Para que "**Eventify**" sea una plataforma real, no podemos depender de la memoria temporal. El negocio requiere que la información de los eventos y lugares se guarde de forma permanente. Además, ante el crecimiento esperado de la oferta, es vital que el sistema permita actualizar datos (ej. cambiar el nombre de un evento) o darlos de baja, y que los listados sean eficientes mediante el uso de páginas para no saturar al usuario ni al servidor.

Detalle de la HU

- **Como:** Administrador del catálogo
- **Quiero:** Gestionar el ciclo de vida completo de los registros en una base de datos real
- **Para:** Garantizar que la información sea persistente, editable y fácil de consultar a gran escala.

TASK 1

Modelado de Entidades y Repositorios (JPA)

1. **Mapeo de Entidades:** Transformar los POJOs en entidades JPA usando `@Entity` y `@Table`.
2. **Identificadores:** Configurar claves primarias con `@Id` y estrategias de generación automática (`@GeneratedValue`).
3. **Restricciones de Columna:** Aplicar `@Column(nullable = false, length = ...)` para asegurar la integridad desde la base de datos.
4. **Repositorios:** Crear interfaces que hereden de `JpaRepository`. Implementar Consultas Derivadas (Derived Queries) como `findByNombreContaining`.

TASK 2

Ciclo de Vida CRUD y Gestión de Errores

1. **Actualización (PUT):** Implementar la lógica para modificar registros existentes. Validar que el recurso exista antes de intentar actualizarlo.
2. **Eliminación (DELETE):** Implementar el borrado físico de registros.
3. **Consulta por ID:** Permitir la recuperación de un único recurso de forma eficiente.

4. **Manejo de "No Encontrado":** Asegurar que si un ID no existe, el sistema responda con un código 404 Not Found en lugar de un error genérico del servidor.

TASK 3

Paginación, Ordenamiento y Calidad

1. **Listados Escalables:** Integrar el uso de `Pageable` y `Sort` en los endpoints de listado para permitir consultas como `?page=0&size=10&sort=nombre,asc`.
2. **Documentación de API:** Reflejar en Swagger los parámetros de paginación y los códigos de error (404, 204).
3. **DataJPA Testing:** Implementar pruebas de integración utilizando `@DataJpaTest` para validar que las entidades se guardan y las consultas personalizadas funcionan en el motor de BD.

Criterios de aceptación

Escenario 1: Persistencia Post-Reinicio (Camino Feliz)

- Dado un registro creado exitosamente en la base de datos.
- Cuando la aplicación se apaga y vuelve a encender.
- Entonces el dato debe seguir disponible, garantizando la persistencia real.

Escenario 2: Intento de Acceso a Recurso Inexistente (Camino de Error)

- Dado un ID que no existe en el sistema (ej. `ID: 9999`).
- Cuando se intenta consultar, actualizar o eliminar dicho ID.
- Entonces el sistema debe retornar un 404 Not Found con un mensaje claro para el cliente.

Escenario 3: Paginación de Resultados (Caso de Volumen)

- Dado un catálogo con 50 eventos.
- Cuando el cliente solicita la página 0 con tamaño 5.
- Entonces el sistema debe entregar exactamente los 5 registros correspondientes y los metadatos de paginación (total de páginas, total de elementos).

Escenario 4: Eliminación Exitosa

- Dado un evento que ya no se realizará.
- Cuando el administrador ejecuta el borrado.
- Entonces el sistema debe retornar un 204 No Content, confirmando que el recurso ya no existe.